

SentinelX

Project Architecture, Component Responsibilities & Full Data Flow

SentinelX is a Bangladesh-focused public-safety and consumer-protection demonstration platform. It supports four role-based portals: citizens, police officers, DNCRP consumer-rights officers, and national administrators.

1. System at a glance

```
React / Vite web application (port 3000)
    | fetch('/api/...') + JWT Bearer token
    v
Flask Python REST API (port 5000)
  |-- authentication and role checks
  |-- reporting, SOS, consumer, police, admin routes
  |-- notifications, audit logs, geocoding, chatbot, analytics
    v
SQLAlchemy database layer
  |-- PostgreSQL when configured
  '-- SQLite file: sentinelx.db in local/demo mode
```

The frontend lives in **src/**. The Python backend lives in **backend/**. **app.py** is the server entry point. During development, Vite proxies every `/api` call to Flask; in a production-style run Flask can serve the built `dist/` frontend itself.

2. Startup sequence

- 1. **Frontend:** `npm run dev` launches Vite at port 3000.
- 2. **Backend:** `python app.py` creates Flask at port 5000.
- 3. **Database:** Flask initializes tables and attempts a PostgreSQL connection. If unavailable, it falls back to local SQLite.
- 4. **Seed data:** On an empty database, demo users, reports, alerts, and barcode data are added.
- 5. **UI:** React starts as a guest, shows the landing page, and opens a dashboard after successful authentication.

3. Frontend responsibilities

Part	What it does
src/App.tsx	Top-level UI controller. Renders the landing page or the correct dashboard according to the authenticated user role.
src/context/AuthContext.tsx	Holds current user, token, alerts, notifications, login/logout, and periodic refresh logic.
src/services/api.ts	One API gateway for all frontend requests. Adds the JWT Authorization header and converts API errors into UI errors.
CitizenDashboard	Crime reports, consumer complaints, SOS, barcode verification, case tracking, AI copilot.
PoliceDashboard	Crime queue, assignment and investigation work, heatmaps, emergency broadcasts, SOS dispatch.

ConsumerDashboard	DNCRP complaint intake, inspections, enforcement, fines, shop/barcode management, rewards.
AdminDashboard	Nationwide overview, user provisioning, audit logs, report export, AI predictions, operational directives.

After login, the browser keeps the JWT in **sessionStorage**. Alerts and notifications are refreshed immediately and then polled every 15 seconds. The project does not use WebSockets for real-time updates.

4. Authentication and access control

```
Register / Login form
  → NID validation and optional email OTP validation
  → User record + bcrypt password hash
  → Flask issues signed JWT
  → Browser saves JWT in sessionStorage
  → React receives user role and opens matching dashboard
```

Protected API requests include Authorization: Bearer <token>. Backend middleware verifies the signed JWT, retrieves the current user from the database, and blocks roles that are not allowed to use that endpoint. The admin uses a separate clearance flow and clearance key.

NID verification defaults to deterministic mock data for the demonstration. It can call Porichoy only if its API configuration is provided. SMTP settings similarly determine whether real emails can be sent.

5. Crime report / GD data flow

```
Citizen crime-report form
  → POST /api/citizen/reports
  → validate mandatory information
  → create report ID and CR tracking ID
  → normalize GPS / location coordinates
  → save CrimeReport, evidence JSON, initial timeline
  → find police officers for the selected Thana
  → create notifications for citizen and police queue
  → write audit log

Police dashboard
  → GET eligible reports
  → verify/reject, claim, or assign a report
  → update status, assigned officer, timeline and investigation notes
  → create citizen notification / optional email and audit record

Citizen dashboard
  → GET own reports
  → view status, assigned officer, evidence, docket and messages
```

The jurisdiction service matches report Thana/district to officer postings. Its officer-selection logic considers jurisdiction, crime type, severity, police specialization, workload, and proximity.

6. Consumer complaint / DNCRP data flow

Citizen consumer-complaint form

- POST /api/citizen/complaints
- create complaint ID and DNCRP tracking number
- save shop, product, barcode, MRP, amount paid, evidence and timeline
- notify citizen and matching DNCRP intake officers
- create audit log

DNCRP dashboard

- claim intake case
- update workflow queue, investigation notes and status
- record final finding, penalty, fine and reward details
- notify citizen

Citizen dashboard

- track complaint and reward status
- download an e-check PDF when eligible

Complaints support shop details, trade licence/BIN, product data, barcode, overcharging information, evidence, officer assignment, investigation summary, final finding, fine, and reward-payment reference.

7. SOS emergency data flow

Citizen presses SOS

- POST /api/citizen/sos with GPS or landmark
- geocoding service resolves reliable coordinates
- jurisdiction service selects appropriate police station
- SOS request stored and police notified
- police dashboard reads SOS queue
- officer records response status, unit and notes
- citizen sees progress or can resolve / stand down the SOS

The app also includes demonstration handling for a store-and-forward style offline SOS packet. It remains a local demo workflow, not a production emergency-dispatch integration.

8. Barcode verification data flow

Citizen enters/scans a barcode

- GET /api/citizen/barcode/<barcode>
- search local barcode registry
- return product, company, BSTI standard, MRP and authenticity status
- optionally use result to prefill a consumer complaint

The frontend also contains an optional Open Food Facts fallback lookup. The main backend registry is the project's local demonstration product catalogue.

9. Messaging, alerts, notifications and auditing

Feature	Flow
Case chat	Authorized citizen/officer sends a message to <code>/api/cases/<case_id>/messages</code> ; Flask stores a <code>CaseMessage</code> record and the other party retrieves it in the dashboard.
Notifications	Case, complaint, reward, SOS, and similar actions create per-user notification rows. The frontend fetches them every 15 seconds and lets a user mark them read.
Emergency broadcasts	Police can create/toggle public emergency alerts. Logged-in users retrieve active alerts and see a banner.
Audit logs	Important actions—report filing, assignment, status changes, exports, and blocked access attempts—write <code>AuditLog</code> records for admin review.

10. Admin and AI analytics flow

```
Admin clearance login
→ AdminDashboard
→ system overview / users / reports / audit logs
→ optional CSV export action recorded in audit log
→ request AI prediction generation
→ prediction engine reads crime-report data + demonstration baseline
→ return risk forecasts, hotspots and suggested directives
→ admin may issue operational directive to police
```

The prediction system is a demonstration analytics engine. It combines seeded/synthetic assumptions with current project reports and can use scikit-learn when installed. It is not a validated operational crime-prediction product.

11. Sentinel Prime chatbot flow

```
Citizen message
→ POST /api/citizen/assistant
→ load the citizen's recent reports, complaints and active alerts
→ match any mentioned case number, barcode or shop
→ extract intent and generate UI action cards
  (SOS prompt, case status, barcode result, reward calculator, form prefill)
→ try Gemini model cascade when API key is configured
→ otherwise return deterministic local legal/help response
```

This means the chatbot is hybrid: it can use Gemini, but its case lookup and guided UI actions have a local rules-based fallback.

12. Core database data

Table / model	Purpose
users	Citizen, police, DNCRP, and administrator accounts; identity, role, posting and credentials.
crime_reports	Crime/GD details, place, evidence, assignment, status and investigation updates.

consumer_complaints	DNCRP complaint, shop/product information, workflow, finding, fine and reward data.
sos_requests	Emergency location, routed station, response unit and SOS status.
emergency_alerts	Police-created public safety broadcasts.
barcode_registry / shops	Product authenticity/MRP records and merchant compliance/reputation information.
notifications / case_messages	Per-user updates and case-specific communication.
audit_logs	Security and operational history.
operational_directives	Admin-issued instructions for police activity and threat response.

13. Backend route groups

Base endpoint	Responsibility
/api/health	Service/database health status.
/api/auth	NID checks, registration, login, admin clearance, OTPs, password changes.
/api/citizen	Reports, complaints, SOS, barcode lookup, notifications, reward e-check, assistant.
/api/police	Crime queue, investigation, officer assignment, heatmap, alerts, SOS response.
/api/consumer	DNCRP complaints, officers, shops, barcode management, rewards.
/api/admin	System overview, users, reports, audit logs, analytics and directives.
/api/cases	Case message thread retrieval and sending.

14. Important project limitations

Academic/demo scope: This project should not be used as a real emergency-reporting or government system. It can fall back to a local SQLite file; external identity, email and AI services are optional/mockable; evidence is stored as JSON rather than hardened file storage; alerts are polling-based; and default credentials/secrets must be replaced for any deployment.

Generated from the current SentinelX source code and README.